

V1.0

从 GCN 到 EvolveGCN

用代码证据拆解动态图神经网络的真实增量

研究对象：GCN $\rightarrow$ GCRN $\rightarrow$ EvolveGCN，并以 DynamicGEM 系列作横向比较

作者：Junxiang Zeng 日期：2026-07-15

报告定位

阶段一 · 论文—代码谱系与核心技术增量审计

摘要

这份报告不从论文的公式包装出发，而从可执行代码追问一个更具体的问题：**GCN、GCRN 与 EvolveGCN 之间，跨时间保存的状态究竟是什么？**对四个仓库、三套框架进行语义级定位后，核心变化可以压缩成一条清楚的接线演化：静态 GCN 固定学习一组卷积权重；GCRN 在节点表示外面套循环网络；EvolveGCN 则让循环网络直接演化 GCN 权重。后一个改动把跨时刻状态从依赖节点数的矩阵，换成只依赖特征维度的矩阵，因此能够自然面对节点出现与消失。

一句话结论 EvolveGCN 的代码创新不在于“又加了一个 RNN”，而在于把 RNN 接到了 **GCN 权重** 上。核心实现只有几十行，但它改变了模型对动态图节点集合的依赖方式。

审计范围与证据口径

本报告锁定以下代码来源。行号按分析时的仓库快照记录；不同仓库使用 TensorFlow、Keras 与 PyTorch，不能依赖文件名直接 diff，因此审计以“变量在计算图中的角色”作为共同语言。

角色	仓库/实现	框架	主要证据
静态基线	tkipf/gcn	TensorFlow	layers.py 中图卷积权重与传播式
循环基线	GCRN / GCN-GRU	PyTorch 等	逐时刻 GCN 表示被堆叠后送入 LSTM/GRU
研究对象	IBM/EvolveGCN	PyTorch	egcn_h.py、egcn_o.py、mat_GRU_gate.py
横向比较	DynamicGEM	Keras	邻接矩阵自编码重构与动态图嵌入

证据分成三层：第一层是模型主前向路径，决定“数据究竟怎么流”；第二层是适配模块，如 TopK 与矩阵 GRU，解释可变节点集合如何被固定尺寸运算接住；第三层是训练目标与输出任务，用来区分技术祖先与仅被比较的方法。报告结论优先采用第一层证据。

四步看清算法演化

第一步：原始 GCN——权重训练后固定

在 tkipf/gcn 的 layers.py 中，每一层创建权重矩阵 weights_i。省略偏置、dropout 与激活后，前向计算可以写成：

$$H^{l+1} = \sigma(\tilde{A}H^lW^l)$$

其中 H 是节点特征， W 是训练得到的层权重， A 是当前图的归一化邻接矩阵。一次训练结束后， W 就作为固定参数使用。这个模型可以处理一张图上的节点聚合，却没有任何跨时间状态，也不知道“上一时刻”是什么。

跨时间保存的东西 没有。模型参数 W 在一次前向传播中不随时间步变化，时间也没有进入模型定义。

第二步：GCRN / GCN-GRU——在节点表示外套循环

常见动态图基线先在每个时刻执行一次 GCN，得到节点嵌入 z_t ，再用 torch.stack 把各时刻结果拼成序列并交给 LSTM/GRU。抽象后是：

$$Z_t = \text{GCN}(A_t, X_t; W), \quad S_t = \text{RNN}(Z_t, S_{t-1})$$

第三方 GCRN 实现中的循环状态形状带有 `nNode` 维度，即每个节点有自己的隐藏状态。它表达了“同一个节点随时间如何变化”，但也把跨时间记忆与节点集合绑定在一起：如果节点数或节点身份改变，旧状态就不再能直接对齐新图。

这里需要保留一个证据边界：用于代码形态说明的第三方 GCRN 仓库晚于原论文，适合作为“典型 GCN-RNN 接法”的实现证据，不应冒充原论文的官方逐行实现。因此这一段的实现对应置信度标为中等。

第三步：EvolveGCN——让循环网络演化权重

`egcn_h.py` 的关键路径可以压缩成两步：

```
GCN_weights = evolve_weights(previous_weights, node_embeddings)
node_embs = activation(Ahat @ (node_embs @ GCN_weights))
```

第一步用矩阵 RNN 根据上一时刻的 GCN 权重以及当前节点信息产生新权重；第二步再用新权重执行图卷积。于是跨时刻保存的状态不再是每个节点的 $N \times d$ 表示，而是层权重 $F_{in} \times F_{out}$ ：

模型	跨时刻状态	尺寸依赖	节点增减
GCN	无	—	没有时间建模
GCRN	节点隐藏状态	$N \times d$	需要节点对齐
EvolveGCN	GCN 层权重	$F_{in} \times F_{out}$	当前步可用任意 N_t

节点数 N_t 仍然出现在当前时刻的邻接矩阵与节点特征中，但不再进入跨时刻状态的形状。这就是 EvolveGCN 能处理节点出现、消失与集合变化的直接代码原因。

H 版：TopK 是尺寸适配器

EvolveGCN-H 更新权重时需要读取当前节点嵌入，但节点数会变，而矩阵 GRU 的输入尺寸固定。TopK 模块先对节点打分，选出固定数量的代表节点，并把不足部分补齐，最终形成固定宽度矩阵。它不是一条旁支技巧，而是可变 N_t 与固定权重尺寸之间的接口。

TopK 的真正作用 它不是为了普通意义上的池化精度，而是把“数量会变的节点集合”整理成矩阵 GRU 能接收的固定形状输入。

O 版：只由旧权重生成新权重

EvolveGCN-O 更激进：更新 `GCN_weights` 时不读取当前节点嵌入，只让权重沿时间自主演化。H 版可以理解为“权重状态 + 当前图信号”，O 版则是“权重状态自身的时间模型”。两者共享“演化权重而非节点状态”的总思想，但信息入口不同。

一个值得记录的论文—代码差异

论文叙述常用 LSTM 类比 O 版的权重演化；官方代码实际沿用 `mat_GRU_cell` 与矩阵门控实现。概念层面仍是门控循环演化权重，但具体单元不是字面上的标准 LSTM。这个差异不推翻论文主张，却说明只读论文容易把实现细节想得过于整齐。

第四步：DynamicGEM——同场比较，不是接线祖先

DynamicGEM 的 `sdne_dynamic.py`、`dynAE.py` 等实现主要用自编码器重构邻接结构。典型输入维度直接与 `node_num` 相关，优化目标是重构误差；EvolveGCN 则执行图消息传递，并围绕节点分类/链路预测等任务训练。两者都处理动态图，但方法论路线不同：

路线	核心运算	主要目标	谱系判断
EvolveGCN	动态图上的图卷积 + 权重演化	任务损失	GCN/GCRN 接线演化
DynamicGEM	邻接/结构自编码重构	重构与嵌入损失	横向比较方法

因此本报告把 DynamicGEM 标成 `COMPARES_WITH`，而不是 EvolveGCN 的技术祖先。标题、应用领域或引用关系相近，不等于主计算路径相承。

从代码行到研究判断

核心创新到底有多大

从实现体积看，EvolveGCN 相对典型 GCRN 的核心变化确实只有几十行：把循环单元的输入输出对象由节点嵌入换成 GCN 权重，再增加一层尺寸适配。这个判断承认“代码增量小”，但不等于“研究价值为零”。真正有价值的是那个接线选择带来的不变量：跨时间状态不再依赖 N 。

论文工程量也不能只按核心 diff 计算。为证明这几十行有效，作者仍需完成动态图数据处理、负采样或任务构造、基线对齐、调参、消融、重复实验、图表与论文写作。这些工作支撑可信度，却不是算法定义本身。

这套审计流水线做了什么

- 锁定论文与仓库快照，避免把同名或后来的实现混在一起；
- 为不同框架建立语义统一的代码地图，定位主前向路径、状态形状和损失；
- 把代码实体组织成 EXTENDS、REPLACES、COMPARES_WITH 等关系；
- 对每个核心判断保留文件、行号、变量名和置信度；
- 将“论文声称什么”与“代码实际执行什么”分开记录。

这使得审计结果不是一句“大模型觉得它们相似”，而是一条可以回到源码复核的证据链。

置信度与限制

静态 GCN 与 EvolveGCN 的核心结论来自官方仓库的主路径，置信度高；GCRN 的具体 state 形状来自典型第三方实现，置信度中等；DynamicGEM 的自编码路线来自官方项目代码与论文描述，足以支持“比较而非祖先”的判断。行号会随仓库更新漂移，公开结果同时保留结构化代码地图，方便用变量名和上下文再次定位。

阶段一结论 真正需要从一篇长论文中提取的，不只是“提出了一个新模型”，而是：它改动了哪一个状态、哪一条数据流、哪一个尺寸依赖，以及这些变化解决了什么约束。EvolveGCN 的答案是：演化权重，从而把跨时间状态与节点数量解耦。

可复核材料

项目仓库保存两类结果：`results/stage1/technical_increment_report.md` 给出完整技术增量报告，`results/stage1/repo_maps/` 保存四个仓库的结构化语义地图。复现脚本位于 `code/`。出于版权与安全边界，仓库不重新分发论文全文、原始 PDF、私有 API 配置或本机路径。

本版本回答的是“代码到底改了什么”。下一阶段在不改变本报告任何阶段一正文的前提下，继续回答“这些代码创新在论文中占了多少段，每段承担什么叙事功能”。